Regression – Single Variate

Lab Task – 1 Simple Linear Regression

Dataset: Power_Plant.csv

Temperature	Vaccum	Pressure	Relative_humidity	Energy
14.96	41.76	1024.07	73.17	463.26
25.18	62.96	1020.04	59.08	444.37
5.11	39.4	1012.16	92.14	488.56
20.86	57.32	1010.24	76.64	446.48
10.82	37.5	1009.23	96.62	473.9
26.27	59.44	1012.23	58.77	443.67
15.89	43.96	1014.02	75.24	467.35
9.48	44.71	1019.12	66.43	478.42
14.64	45	1021.78	41.25	475.98

Independent variables

Dependent variables

Predict the energy of the power plant for the given pressure using a regression model.

Note:
It is clear from the question that energy (dependent variable) must be estimated based on pressure (independent variable).

Answer:

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('Power_Plant.csv')`
	`X = dataset.iloc[:, [2]].values`
	`y = dataset.iloc[:, -1].values`
	`print(X)`
	`print(y)`
	`Note: 2 is the index of the pressure column`

c.	Splitting the dataset into the Training set and Test set

	`from sklearn.model_selection import train_test_split`
	`X_train, X_test, y_train, y_test = train_test_split(X, y,`
	`test_size = 1/3, random_state = 0)`

d.	Training the Simple Linear Regression model on the Training set

	`from sklearn.linear_model import LinearRegression`
	`regressor = LinearRegression()`
	`regressor.fit(X_train, y_train)`

e.	Predicting the Test set results

	`y_pred = regressor.predict(X_test)`
	`print(X_test)`
	`Note: it is the predicted value for the energy column`

f.	Predicting the Specific Value

	`# find the energy(dependent variable) for pressure(ind var)=1024.07`
	`regressor.predict([[1024.07]])`
	
	`array([470.68599678])`
	
	`Note:` Therefore, our model predicts that the energy of the power plant is 470.68 when the pressure is 1024.07.
	`Observe that the value of the feature (1024.07) has been enclosed in double square brackets. This is due to the fact that the "predict" method always expects a 2D array as its input format. And placing 1024.07 between double square brackets makes the input a 2D array.`
	`1024.07 →scalar`
	`[1024.07]→1D array`
	`[[1024.07]]→2D array`

g.	Visualising the Training set results

	`plt.scatter(X_train, y_train, color = 'red')`
	`plt.plot(X_train, regressor.predict(X_train), color = 'blue')`
	`plt.title('Pressure vs Energy (Training set)')`
	`plt.xlabel('Presure')`
	`plt.ylabel('Energy')`
	`plt.show()`

h.	Visualising the Test set results

	`plt.scatter(X_test, y_test, color = 'yellow')`
	`plt.plot(X_train, regressor.predict(X_train), color = 'green')`
	`plt.title('Pressure vs Energy (Test set)')`
	`plt.xlabel('Pressure')`
	`plt.ylabel('Energy')`
	`plt.show()`

i.	Getting the final linear regression equation

	`print(regressor.coef_)`
	`print(regressor.intercept_)`
	`[1.50117327]`
	`-1066.6205133949695`
	
	`Therefore, the equation of our simple linear regression model(y=ax+b) is:`
	`Energy = -1066.62051 + 1.5011 * Pressure`

j	Accuracy of the Linear Regression equation using R-squared
	`from sklearn.metrics import r2_score` `print(r2_score(y_test, y_pred))`

	0.2611671708679816 It means only 26% accurate this model possessed to predict the value.

Find the energy of the power plant for the following pressure using above regression equation:

Pressure	Energy
1010.74	
1014.02	
1015.14	

k.	Lasso Regularisation(L1) - To improve the accuracy
	# lasso regularisation from sklearn.linear_model import Lasso las=Lasso(alpha=0.1) las.fit(X_train,y_train) y_pred_test=las.predict(X_test) print(r2_score(y_test,y_pred_test)) 0.26118947934551573 The accuracy is still 26%
l.	Ridge Regularisation(L2) - To improve the accuracy
	# ridge regularisation from sklearn.linear_model import Ridge rdge=Ridge(alpha=0.1) rdge.fit(X_train,y_train) y_pred_test=rdge.predict(X_test) print(r2_score(y_test,y_pred_test)) 0.2611671763495824 The accuracy is still 26%
m.	Elasticnet Regularisation(L1+L2) - To improve the accuracy
	# Elasticnet regularisation from sklearn.linear_model import ElasticNet enet=ElasticNet(alpha=0.1) enet.fit(X_train,y_train) y_pred_test=enet.predict(X_test) print(r2_score(y_test,y_pred_test)) 0.26119472504680685

	The accuracy is still 26%

Regression – Multivariate

Lab Task – 2 Multiple Linear Regaression

Predict the amount of energy to be generated from the power plant using a regression model using the following parameters:
Temperature
Vaccum
Pressure
Relative Humidity

The regression models, such as multiple linear regression and polynomial regression should be applied to the dataset, and the best regression model should be chosen for the given dataset.

Answer:

Multiple Linear Regression:

a.	Importing the libraries

	`import numpy as np`
	`import matplotlib.pyplot as plt`
	`import pandas as pd`

b.	Importing the dataset

	`dataset = pd.read_csv('Power_Plant.csv')`
	`X = dataset.iloc[:, :-1].values`
	`y = dataset.iloc[:, -1].values`
	`print(X)`
	`print(y)`

c.	Splitting the dataset into the Training set and Test set

	`from sklearn.model_selection import train_test_split`
	`X_train, X_test, y_train, y_test = train_test_split(X, y,`
	`test_size = 0.2, random_state = 0)`

d.	Training the Multiple Linear Regression model on the Training set

	`from sklearn.linear_model import LinearRegression`
	`regressor = LinearRegression()`
	`regressor.fit(X_train, y_train)`

e.	Predicting the Test set results
	`y_pred = regressor.predict(X_test)` `np.set_printoptions(precision=2)`

	print(np.concatenate((y_pred.reshape(len(y_pred),1), y_test.reshape(len(y_test),1)),1))
f.	Predicting the Specific Value

	# find the energy(dependent variable) for temp,vaccum,pressure and humidity(ind vars)
	regressor.predict([[14.96,41.76,1024.07,73.1]])
	array([467.31])

g.	Evaluating the Model Performance
	from sklearn.metrics import r2_score r2_score(y_test, y_pred) 0.9325315554761303 Note: Therefore, for the given dataset, R^2 for simple linear regression model is 93.25%
h.	Getting the Final Multiple Linear Regression Equation
	print(regressor.coef_) print(regressor.intercept_) [-1.97 -0.24 0.06 -0.16] 452.84103716163844 Note: Form the output, we can infer the multi linear regression equation is : Energy = 452.84103716163844 -1.97 * temperature -0.24* vacuum 0.06*pressure -0.16*humidity

Lab Task – 3 Polynomial Regression

Predict the energy of the power plant for the given pressure using a regression model.

Polynomial Regression:

a.	Importing the libraries
	import numpy as np import matplotlib.pyplot as plt import pandas as pd
b.	Importing the dataset
	dataset = pd.read_csv('Power_Plant.csv') X = dataset.iloc[:, [2]].values y = dataset.iloc[:, -1].values print(X) print(y)
c.	Splitting the dataset into the Training set and Test set
	from sklearn.model_selection import train_test_split X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)
d.	Training the Polynomial Regression model on the Training set
	from sklearn.preprocessing import PolynomialFeatures

	`from sklearn.linear_model import LinearRegression`
	`poly_reg = PolynomialFeatures(degree = 4)`
	`X_poly = poly_reg.fit_transform(X_train)`
	`regressor = LinearRegression()`
	`regressor.fit(X_poly, y_train)`

e.	Predicting the Test set results

	`y_pred = regressor.predict(poly_reg.transform(X_test))`
	`np.set_printoptions(precision=2)`
	`print(np.concatenate((y_pred.reshape(len(y_pred),1),`
	`y_test.reshape(len(y_test),1)),1))`

f.	Predicting the Specific Value
	`# find the energy(dependent variable) for pressure(ind var)=1024.07` `regressor.predict(poly_reg.fit_transform([[1024.07]]))`
g.	Evaluating the Model Performance

	`from sklearn.metrics import r2_score`
	`r2_score(y_test, y_pred)`
	`0.2960206068764252`
	Note: Therefore, for the given dataset, R^2 for polynomial regression model is 29.60%

	Getting the Final Polynomial Regression Equation

	`print(regressor.coef_)`
	`print(regressor.intercept_)`
	`[ 0.00e+00 -1.11e-02 -7.50e+00  9.84e-03 -3.63e-06]`
	`1289269.0209504422`
	Note: Form the output, we can infer the polynomial equation is :
	Energy = 1289269.0209504422-1.11e-02*pressure^1 -7.50e+00* pressure^2 +9.84e-03*pressure^3 -3.63e-06*pressure^4

From the above,

Model	R^2
Multiple Linear	93.25%
Polynomial	29.60%

Thus, Multiple linear regression model is suitable for the given dataset (Power_Plant).

Do it yourself:

Develop and compare regression models to predict the 'Total' marks based on 'Quiz', 'Practical', and 'Project' scores. Implement Multiple Linear Regression, Polynomial Regression, Lasso, Ridge, and Elastic Net models. Select the best-fitting model based on evaluation metrics. Analyze the model's performance by comparing predicted 'Total' marks with the actual 'Total' marks in the dataset.

Steps to be followed:

1. Loading and Preprocessing the Data

- **Load Dataset**:
  - The dataset (`Marks.csv`) is loaded using `pandas`.
  - Relevant columns (`Quiz`, `Practical`, `Project`, `Final`) are extracted for analysis.
- **Handle Missing Values**:
  - Missing values in the selected columns are replaced with their respective column means to ensure the dataset is complete and usable for modeling.
- **Create Target Variable**:
  - A new column, `Total`, is computed as a weighted sum of the independent variables:
    - `Total = 0.1 * Quiz + 0.3 * Practical + 0.3 * Project + 0.3 * Final`.

2. Detecting and Removing Outliers

- **Outlier Detection**:
  - For each independent variable (`Quiz`, `Practical`, `Project`, `Final`), outliers are identified using the **Interquartile Range (IQR) Method**:
    - Values outside `[Q1 - 1.5*IQR, Q3 + 1.5*IQR]` are flagged as outliers.
- **Combine Outliers**:
  - Outliers detected for all variables are consolidated into a single DataFrame and duplicates are removed.
- **Remove Outliers**:
  - Rows corresponding to the detected outliers are removed from the dataset to improve model performance.

3. Splitting the Data

- **Independent Variables (X)**:
  - The cleaned dataset's independent variables: `Quiz`, `Practical`, `Project`, and `Final`.
- **Target Variable (y)**:
  - The target variable `Total`.

- **Train-Test Split**:
  - The dataset is split into training (80%) and testing (20%) sets using `train_test_split`.

4. Applying Regression Models

- **Regression Models**:
  - Multiple regression models are applied to predict the target variable `Total`:
    1. **Linear Regression**:
       - Fits a straight line to the data.
    2. **Polynomial Regression (Degree 2)**:
       - Transforms features to include polynomial terms of degree 2.
    3. **Ridge Regression**:
       - Linear regression with L2 regularization to reduce overfitting.
    4. **Lasso Regression**:
       - Linear regression with L1 regularization for feature selection.
    5. **Elastic Net Regression**:
       - Combines L1 and L2 regularization.
- **Polynomial Features**:
  - For polynomial regression, the features are transformed to include polynomial terms using `PolynomialFeatures`.

5. Evaluating Models

- **Predictions**:
  - Each model predicts the target variable (`Total`) for the test set.
- **Evaluation Metrics**:
  - **Mean Squared Error (MSE)**:
    - Measures the average squared difference between predicted and actual values.
  - **R-squared (R^2)**:
    - Indicates how well the model explains the variability of the target variable.

6. Selecting the Best Model

- **Comparison Table**:
  - A DataFrame is created to compare all models based on MSE and R^2.
- **Best-Fit Model**:
  - The model with the highest R^2 value is selected as the best-fit model.

7. Printing the Regression Equation

- **For Polynomial Regression**:
  - Coefficients and intercept are used to construct the regression equation, including polynomial terms.
- **For Other Models**:
  - Coefficients of the linear terms and the intercept are printed.

8. Visualization

- **Observed vs. Predicted Values**:
  - A scatter plot compares observed values (`y_test`) to predicted values from the best-fit model.
  - A red diagonal line represents perfect predictions.

Sample Python code:

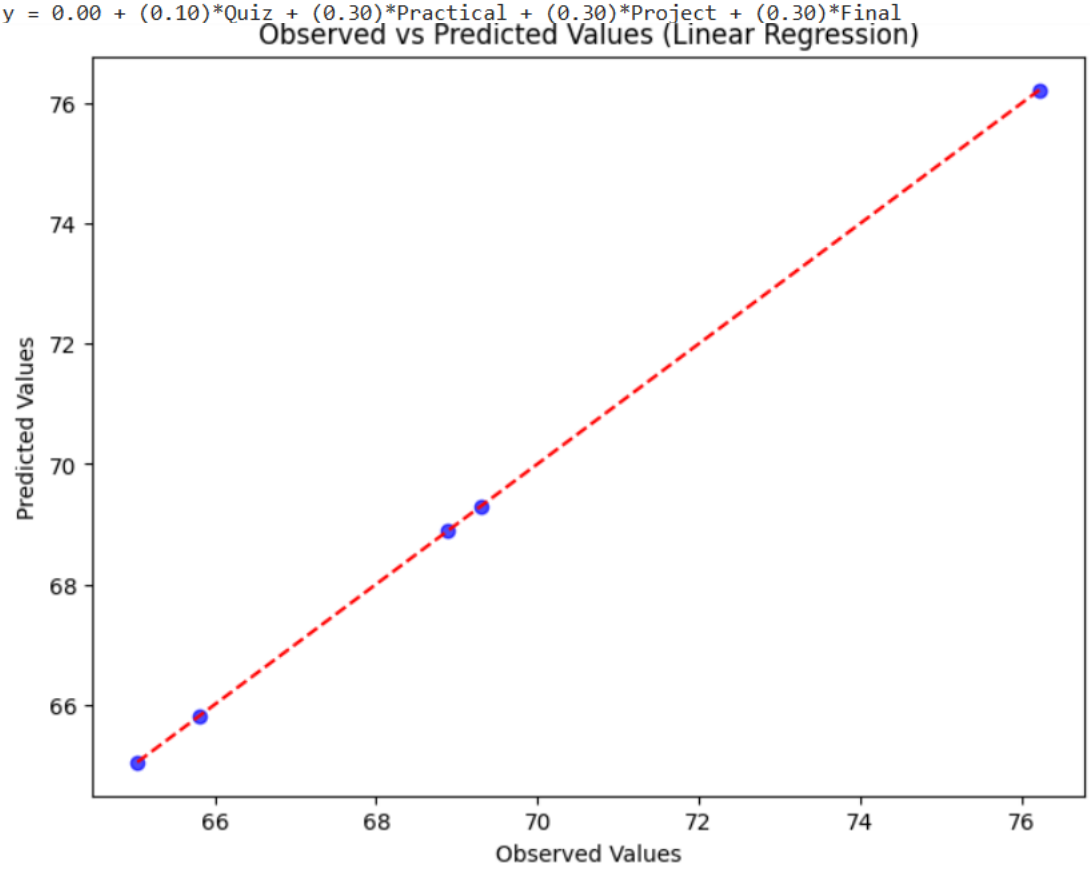
```
# Importing necessary libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.preprocessing import PolynomialFeatures
from sklearn.metrics import mean_squared_error, r2_score

# Load the dataset
marks_file_path = 'Marks.csv'  # Update the path to your dataset
marks_data = pd.read_csv(marks_file_path)

# Select relevant columns and preprocess the data
filtered_data = marks_data[['Quiz', 'Practical', 'Project', 'Final']].copy()

# Fill missing values with column means
filtered_data = filtered_data.fillna(filtered_data.mean())

# Create a new column 'Total' as the target variable
filtered_data['Total'] = (
    0.1 * filtered_data['Quiz'] +
    0.3 * filtered_data['Practical'] +
    0.3 * filtered_data['Project'] +
    0.3 * filtered_data['Final']
)

# Detect Outliers using the IQR method
def detect_outliers(data, column):
    Q1 = data[column].quantile(0.25)
    Q3 = data[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return data[(data[column] < lower_bound) | (data[column] > upper_bound)]

# Detect outliers for all columns
outliers_list = []
```

```
for col in ['Quiz', 'Practical', 'Project', 'Final']:
    outliers_list.append(detect_outliers(filtered_data, col))

outliers = pd.concat(outliers_list).drop_duplicates()

# Remove outliers from the dataset
filtered_data_no_outliers = filtered_data[~filtered_data.index.isin(outliers.index)]

# Define independent variables (X) and target variable (y)
X = filtered_data_no_outliers[['Quiz', 'Practical', 'Project', 'Final']]
y = filtered_data_no_outliers['Total']

# Split the dataset into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=1)

# Models to compare
models = {
    "Linear Regression": LinearRegression(),
    "Polynomial Regression (Degree 2)": LinearRegression(),
    "Ridge Regression": Ridge(alpha=1.0),
    "Lasso Regression": Lasso(alpha=0.1),
    "Elastic Net Regression": ElasticNet(alpha=0.1, l1_ratio=0.5)
}

# Polynomial Features for Polynomial Regression
poly_features = PolynomialFeatures(degree=2)
X_poly_train = poly_features.fit_transform(X_train)
X_poly_test = poly_features.transform(X_test)

# Results storage
results = []
predicted_values = {}

# Evaluate each model
for name, model in models.items():
    if "Polynomial" in name:
        model.fit(X_poly_train, y_train)
        y_pred = model.predict(X_poly_test)
    else:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    r2 = r2_score(y_test, y_pred)
    results.append({"Model": name, "Mean Squared Error": mse, "R-squared": r2})
    predicted_values[name] = y_pred
```

```
# Create a DataFrame for results
comparison_results = pd.DataFrame(results)

# Identify the best-fit model based on R-squared
best_model_name = comparison_results.loc[comparison_results['R-squared'].idxmax(),
'Model']
best_model_r2 = comparison_results.loc[comparison_results['R-squared'].idxmax(), 'R-
squared']
best_model_mse = comparison_results.loc[comparison_results['R-squared'].idxmax(),
'Mean Squared Error']

print("Comparison of Regression Models:")
print(comparison_results)
print(f"\nBest-Fit Model: {best_model_name}")
print(f"R-squared: {best_model_r2}")
print(f"Mean Squared Error: {best_model_mse}")

# Print Regression Equation for the Best-Fit Model
if best_model_name == "Polynomial Regression (Degree 2)":
    coefficients = models["Polynomial Regression (Degree 2)"].coef_
    intercept = models["Polynomial Regression (Degree 2)"].intercept_
    feature_names = poly_features.get_feature_names_out(X.columns)
    print("\nPolynomial Regression Equation (Degree 2):")
    equation = f"y = {intercept:.2f} "
    for coef, feature in zip(coefficients, feature_names):
        equation += f"+ ({coef:.2f})*{feature} "
    print(equation)
else:
    best_model = models[best_model_name]
    intercept = best_model.intercept_
    coefficients = best_model.coef_
    print(f"\nRegression Equation for {best_model_name}:")
    equation = f"y = {intercept:.2f} "
    for coef, feature in zip(coefficients, X.columns):
        equation += f"+ ({coef:.2f})*{feature} "
    print(equation)

# Plot Observed vs. Predicted values for the best model
plt.figure(figsize=(8, 6))
plt.scatter(y_test, predicted_values[best_model_name], alpha=0.7, color="blue")
plt.plot([min(y_test), max(y_test)], [min(y_test), max(y_test)], color='red', linestyle="--")
plt.title(f"Observed vs Predicted Values ({best_model_name})")
plt.xlabel("Observed Values")
plt.ylabel("Predicted Values")
plt.show()
```

Output:

```
Comparison of Regression Models:
                             Model  Mean Squared Error  R-squared
0                Linear Regression        0.000000e+00   1.000000
1  Polynomial Regression (Degree 2)        1.684250e-26   1.000000
2                 Ridge Regression        1.750587e-05   0.999999
3                 Lasso Regression        4.692068e-04   0.999970
4            Elastic Net Regression        2.068225e-04   0.999987

Best-Fit Model: Linear Regression
R-squared: 1.0
Mean Squared Error: 0.0

Regression Equation for Linear Regression:
y = 0.00 + (0.10)*Quiz + (0.30)*Practical + (0.30)*Project + (0.30)*Final
```



Additional Regression Model (Optional):

Try other regression model using ChatGPT for the power plant dataset and choose the best.

Power_Plant.csv
Spreadsheet

Write Python code to create a regression model like multiple linear, polynomial, ridge, lasso, elastic net, and gradient boosting for the attached dataset. The program should include printing the r2_score, regression intercept, and coefficient for each model. It should also include the code to predict the values for each model and print the predicted value.

After executing the above,

provide the Python code

Output:

Model	R2 Score	Intercept	Coefficients	First 5 Predictions
Linear Regression	0.930104643	454.5691146	[-1.98589969 -0.23209358 0.06219991 -0.15811779]	[455.68020791 438.73212215 434.16444 438.76954566 479.88832866]
Polynomial Regression	0.938277671	-7401.885922	[0.00000000e+00 -5.85369923e+00 -2.74971280e+00 1.54111904e+01 3.95258752e+00 1.80224460e-02 1.03543609e-02 3.14522800e-03 -5.52783968e-03 -6.09799738e-04 2.27772424e-03 6.73015572e-05 -7.50140811e-03 -3.63966215e-03 -1.92289852e-03]	[454.98956396 439.05053394 434.9290992 438.84398037 480.6131461]
Ridge Regression	0.930104717	454.5621089	[-1.98587284 -0.23210375 0.06220652 -0.15811329]	[455.680289 438.73205978 434.16450255 438.76947183 479.8882337]

Lasso Regression	0.930124157	455.932999	[-1.98086437 -0.23373688 0.06073755 -0.15664236]	[455.69059249 438.70527233 434.19685571 438.73713016 479.84565075]
Elastic Net Regression	0.93013693	452.7226325	[-1.97364566 -0.2366149 0.0638558 -0.15575211]	[455.71484949 438.69596054 434.2032558 438.72644878 479.83266184]
Gradient Boosting Regression	0.949460972			[455.22612584 436.47687063 432.91326329 435.88088525 478.7227488]

```
                         Model  R2 Score    Intercept  \
0             Linear Regression  0.930105   454.569115
1          Polynomial Regression  0.938278 -7401.885922
2              Ridge Regression  0.930105   454.562109
3              Lasso Regression  0.930124   455.932999
4        Elastic Net Regression  0.930137   452.722632
5  Gradient Boosting Regression  0.949461          NaN

                                      Coefficients  \
0  [-1.98589969477556, -0.23209357748432818, 0.06...
1  [0.0, -5.853699226606111, -2.749712797959006, ...
2  [-1.985872843124843, -0.23210375188612567, 0.0...
3  [-1.9808643708071891, -0.23373687551045447, 0....
4  [-1.973645664824961, -0.23661489643001912, 0.0...
5                                               None

                                First 5 Predictions
0  [455.6802079112853, 438.73212214727334, 434.16...
1  [454.9895639631741, 439.0505339384872, 434.929...
2  [455.6802890015872, 438.73205978296096, 434.16...
3  [455.69059249378284, 438.7052723349901, 434.19...
4  [455.7148494850332, 438.69596054421334, 434.20...
5  [455.2261258387762, 436.4768706325861, 432.913...
```